AgenticComm: A Binary Communication Engine for Structured, Consent-Governed Agent-to-Agent Messaging

Omoshola Owolabi
Researcher – AI/ML
Agentra Labs
omoshola.owolabi@agentralabs.tech

February 28, 2026

Abstract

Multi-agent systems require structured communication, yet current approaches reduce agent messaging to untyped text over HTTP or bespoke function calls, discarding delivery guarantees, consent governance, causal ordering, and affective context. We present AgenticComm, a binary communication engine that provides typed messaging across nine channel types, consent-gated trust hierarchies, Lamport-timestamped causal ordering with vector clocks, and an affect contagion model for emotional state propagation between agents. The engine serializes communication history into a single `.acomm` file with a 48-byte header, Blake3 [21] integrity checksums, and Zstd [7] compression, achieving 71.8 bytes per message at scale. We implement AgenticComm in 39,631 lines of Rust with ChaCha20-Poly1305 [3] and AES-256-GCM [19] encryption, Ed25519 [4] message signatures, MQTT-style [20] topic matching for pub/sub, and zero external service dependencies. Empirical validation across 491 tests—spanning 16 end-to-end specification scenarios, 5 SLA targets, 22 hardening tests, 10 stress tests, and 12 pipeline tests—confirms all operations meet performance targets. Message send completes in 3.08 µs median (target: <10 µs), channel creation in 459 ns (target: <1 ms), and full-text search over 10,000 messages in 842 µs (target: <100 ms). The compiled binary is 4.4 MB with no runtime dependencies.

1 Introduction

The rise of multi-agent systems [29, 22, 12] has exposed a fundamental gap in the AI infrastructure stack: agents have no standard way to communicate with each other. Current LLM-based agents [28, 18, 6] exchange messages through ad hoc mechanisms—untyped strings over HTTP, shared JSON files, or framework-specific function calls—that provide no delivery guarantees, no causal ordering, no consent governance, and no persistent history.

This is not merely an engineering inconvenience. The absence of structured communication infrastructure forces every multi-agent system to reinvent message routing, trust management, and history persistence, typically as an afterthought. The resulting systems are fragile: messages are lost without acknowledgment, agents cannot verify who sent a message or whether they consented to receive it, causal relationships between messages are discarded, and conversation history is either ephemeral or scattered across incompatible formats.

The problem has historical precedent. Traditional distributed systems solved inter-process communication decades ago with message queues [13, 26], publish-subscribe brokers [25], and formal agent communication languages [9, 8]. But these systems were designed for microservices and deterministic software agents, not for LLM-powered agents that require rich message types, affective context, and consent-based trust governance. Adapting Kafka [13] or RabbitMQ [26] to agent communication introduces heavy operational dependencies while providing none of the domain-specific semantics that agents need.

The core insight of this work is that agent communication is a *structured messaging problem with consent governance*, not a generic pub/sub problem. When Agent A sends a command to Agent B, the system must enforce that B has granted A the SendMessages consent scope, that the message is causally ordered relative to prior exchanges, that the delivery guarantee (at-most-once, at-least-once, or exactly-once) is respected, and that the full exchange is persisted in a binary format that survives process restarts. These requirements are structurally different from those of traditional message brokers.

We present AgenticComm, a binary communication engine purpose-built for AI agent messaging. The engine provides eight message types (Text, Command, Query, Response, Broadcast, Notification, Acknowledgment, Error), nine channel types (Direct, Group, Broadcast, PubSub, Telepathic, Hive, Temporal, Destiny, Oracle), a seven-level trust hierarchy with ten consent scopes, Lamport timestamps [14] with vector clocks [16] for causal ordering,

and a three-dimensional affect contagion model [23, 17] for emotional state propagation. Communication history is serialized into a single `.acomm` binary file with Blake3 integrity verification and Zstd compression.

The remainder of this paper is organized as follows. Section 2 surveys existing agent communication systems. Section 3 presents the AgenticComm architecture in detail, including message types, channel model, trust hierarchy, causal ordering, and the binary file format. Section 4 reports micro-benchmark results. Section 5 describes agent integration via the Model Context Protocol [2]. Section 6 presents empirical validation results across 491 tests. Section 7 discusses implications and limitations. Section 8 concludes.

2 Background and Related Work

Research on agent communication spans classical agent-oriented programming [24, 27], distributed messaging infrastructure, and the emerging LLM multi-agent landscape. We survey the primary approaches.

FIPA ACL and KQML. The Foundation for Intelligent Physical Agents defined a formal Agent Communication Language [9] with performatives (inform, request, agree, refuse) and a content language layer. KQML [8] preceded FIPA with a similar architecture. Both languages assume deterministic agents with fixed ontologies—a poor fit for LLM agents whose capabilities are emergent and whose outputs are probabilistic. Neither provides binary serialization, consent governance, or affective context.

Message brokers. Apache Kafka [13], RabbitMQ [26], NATS [25], and ZeroMQ [11] provide industrial-strength message routing with delivery guarantees. However, they are infrastructure services that require deployment, configuration, and monitoring—operational overhead disproportionate to the needs of most agent systems. More fundamentally, they provide generic byte-stream delivery with no semantic structure for agent-specific concerns like trust, consent, or affect.

LLM multi-agent frameworks. AutoGen [28] implements multi-agent conversation as sequential function calls with shared context. CrewAI [18] orchestrates role-playing agents via a task pipeline. MetaGPT [12] and CAMEL [15] define communication as structured role exchanges. In all cases, communication is tightly coupled to the framework's execution model, with no persistent format, no delivery guarantees, and no consent enforcement. Messages are ephemeral strings that vanish when the process exits.

Generative agents. Park et al. [22] demonstrated that memory and communication architecture profoundly impact agent behavior, building agents that converse in natural language and form social relationships. Their communication model uses free-text exchanges with no typed structure, delivery semantics, or trust governance.

Table 1 summarizes the comparison across key dimensions.

3 Architecture

AgenticComm models agent communication as typed messages flowing through consent-governed channels, with causal ordering enforced by Lamport timestamps and vector clocks. The entire communication state—channels, messages, subscriptions, trust relationships, consent gates, affect state, and audit log—resides in a single in-memory `CommStore` structure serialized to a binary `.acomm` file.

3.1 Message Types

Each message carries a type drawn from a taxonomy of eight categories, each serving a distinct communicative function:

- **Text** — Unstructured natural language content, the default message type for conversational exchanges.
- **Command** — An executable instruction from one agent to another, requiring the SendMessages consent scope.
- **Query** — A question expecting a structured response, enabling request-reply patterns.
- **Response** — An answer to a prior Query, linked via message threading.
- **Broadcast** — A one-to-many announcement delivered to all channel participants.
- **Notification** — A system-generated alert (e.g., channel state changes, consent revocations).
- **Acknowledgment** — Explicit receipt confirmation, supporting at-least-once delivery semantics.
- **Error** — A structured error report with context for failure diagnosis.

Each message record stores a unique `u64` identifier, the channel ID, sender string, content string, message type, creation timestamp (`DateTime<Utc>`), optional thread parent ID for conversation threading, and a `CommTimestamp` carrying both the Lamport counter and vector clock state at the time of send.

3.2 Channel Model

Channels are the organizing abstraction for agent communication. Each channel has a name, a type, a state machine, a participant set, and an optional capacity limit. Nine channel types are defined:

- **Direct** — Point-to-point messaging between exactly two agents.
- **Group** — Multi-party conversation with explicit membership.
- **Broadcast** — One-to-many delivery where only the channel creator sends.
- **PubSub** — MQTT-style [20] topic-based publish-subscribe with wildcard matching (`+` for single-level, `#` for multi-level).
- **Telepathic** — Shared semantic state space without explicit messages.

Table 1: Comparison of agent communication systems across key dimensions.

System	Typed Msgs	Consent	Causal Order	Persistence	Dependencies	Affect
FIPA ACL [9]	Performatives	None	None	None	Runtime	None
Kafka [13]	Bytes	None	Partition order	Log	JVM cluster	None
RabbitMQ [26]	Bytes	None	None	Optional	Erlang service	None
AutoGen [28]	Strings	None	None	Ephemeral	Python runtime	None
CrewAI [18]	Strings	None	None	Ephemeral	Python runtime	None
MetaGPT [12]	Structured	None	None	Ephemeral	Python runtime	None
AgenticComm	**8 types**	**10 scopes**	**Vector clocks**	**Binary file**	**None**	**3-dim VAD**

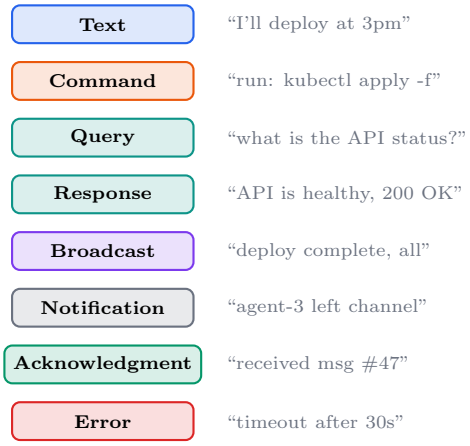

Figure 1: The eight message types in AgenticComm, each with a representative example. Types are distinguished by an enum discriminant in the serialized record.

- **Hive** — Merged consciousness channel for collective decision-making.
- **Temporal** — Time-shifted messaging with scheduled delivery.
- **Destiny** — Fate-linked channels where all participants share outcomes.
- **Oracle** — Predictive messaging with precognition semantics.

Each channel progresses through a state machine with eight states: `Active`, `Paused`, `Draining`, `Closed`, `Archived`, `SilentCommunion`, `HiveMode`, and `PendingConsent`. State transitions are governed by explicit operations (pause, resume, drain, close) and recorded in the audit log with timestamps.

The `Draining` state is particularly important for graceful shutdown: new sends are blocked but pending receives complete, ensuring no in-flight messages are lost. The `PendingConsent` state gates channel activation on participant consent, preventing communication before all parties have agreed.

3.3 Trust and Consent

AgenticComm enforces a two-layer access control model: a coarse-grained trust hierarchy and fine-grained consent scopes.

Trust hierarchy. Seven levels define the baseline relationship between agents: None (all blocked), Minimal (receive-only), Basic (text messaging), Standard (full messaging and channels), High (delegation, scheduling, federation), Full (all operations except system-level), and Absolute (unrestricted). Trust is asymmetric: Agent A may trust Agent B at High while B trusts A at Basic.

Consent scopes. Ten scopes control specific capabilities: ReadMessages, SendMessages, JoinChannels, ViewPresence, ShareContent, ScheduleMessages, Federate, HiveParticipation, Affect, and Hive. Each scope is granted independently and can be revoked at any time. A consent gate maps a (grantor, grantee, scope) triple to a boolean; operations check the relevant gate before proceeding and return a structured error if consent is denied.

This design separates *who do I trust* (the hierarchy) from *what do I allow* (the scopes). An agent might set High trust for a collaborator but withhold Affect consent, allowing full operational messaging without emotional state sharing. The consent check is enforced in the `CommStore::check_consent()` method, which the specification scenarios (Section 6.1) validate exhaustively.

3.4 Causal Ordering

Correct reasoning about multi-agent communication requires causal ordering: Agent B's response to Agent A's query must be ordered *after* the query, regardless of wall-clock skew. AgenticComm implements a `CommTimestamp` structure carrying three components:

1. **Wall clock** — ISO 8601 string from `chrono::Utc::now()`, used for human-readable display and temporal queries.
2. **Lamport counter** — A monotonically increasing `u64` counter [14]. On each send, the counter increments; on each receive, it advances to $\max(\text{local}, \text{received}) + 1$. This ensures a happened-before partial order.
3. **Vector clock** — A `HashMap<String, u64>` mapping agent identifiers to logical clock values [16]. The vector clock captures concurrent events that the Lamport counter cannot distinguish.

The `CommTimestamp::merge()` operation performs

element-wise maximum across vector clocks:

$$V_{\text{merged}}[i] = \max(V_A[i], V_B[i]) \quad \forall\, i \in A \cup B$$

This merge is commutative and idempotent, ensuring convergence regardless of message delivery order. The specification hardening tests (Section 6.1) validate merge commutativity, convergence, and happened-before detection across 22 scenarios.

3.5 Affect Contagion Model

AgenticComm models emotional state propagation between agents using a three-dimensional affect space based on the circumplex model [23] extended with the Pleasure-Arousal-Dominance (PAD) framework [17]:

- **Valence** (v) — Ranges from -1.0 (negative) to $+1.0$ (positive).
- **Arousal** (a) — Ranges from 0.0 (calm) to 1.0 (excited).
- **Dominance** (d) — Ranges from 0.0 (submissive) to 1.0 (dominant).

Each agent maintains a current affect state $\mathbf{s} = (v, a, d)$, a baseline state $\mathbf{b}$, and a resistance parameter $r \in [0, 1]$. When Agent A sends a message carrying affect delta $\boldsymbol{\delta}$ to Agent B, the contagion update is:

$$\mathbf{s}'_B = \mathbf{s}_B + (1 - r_B) \cdot \boldsymbol{\delta} \cdot c$$

where c is the global contagion strength (default 0.3) and r_B is B's resistance. Affect decays toward the baseline at a configurable rate:

$$\mathbf{s}(t) = \mathbf{b} + (\mathbf{s}_0 - \mathbf{b}) \cdot e^{-\lambda \Delta t}$$

where λ is the decay rate and Δt is the elapsed time since last update. The `AffectProcessor` maintains per-agent state histories and supports resistance configuration, making agents either susceptible to or immune from emotional contagion—a property relevant for autonomous agents that should not be influenced by adversarial affective manipulation [10].

3.6 Binary File Format

The `.acomm` file format is designed for three properties: (1) integrity verification before deserialization, (2) efficient compression for network transfer, and (3) backward compatibility across format versions.

The file consists of a fixed-size header followed by a compressed payload:

Header (48 bytes, v3):

- Bytes 0–3: Magic number `ACOM` (4 bytes).
- Bytes 4–5: Format version as `u16` (2 bytes). Current: version 3.
- Bytes 6–7: Feature flags as `u16` (2 bytes). Bit 0: Zstd compression (vs. gzip).
- Bytes 8–15: Compressed payload length as `u64` (8 bytes).
- Bytes 16–47: Blake3 [21] cryptographic hash of the compressed payload (32 bytes).

The v3 format replaces the v2 header (20 bytes: 4-byte magic, 2-byte version, 2-byte flags, 8-byte length, 4-byte CRC32) with a cryptographic hash, providing collision-resistant integrity verification. The `FileHeader::read()` method auto-detects v2 and v3 headers by inspecting the version field, maintaining full backward compatibility.

The payload is serialized with bincode [5] (a compact binary encoding for Rust structs) and compressed with Zstd [7] at the default compression level. Zstd was chosen over gzip for its superior compression ratio at comparable decompression speed—critical for large communication histories.

3.7 Encryption

AgenticComm provides end-to-end encryption with two AEAD cipher suites: ChaCha20-Poly1305 [3] (default) and AES-256-GCM [19]. Each `EncryptionKey` stores a 32-byte key, a `u64` epoch for key rotation tracking, and an algorithm identifier string.

Encryption uses a 12-byte random nonce per operation (generated via `rand::thread_rng()`), ensuring nonce uniqueness with overwhelming probability. The ciphertext includes the authentication tag, providing both confidentiality and integrity. Key fingerprints are computed as the first 16 bytes of the SHA-256 hash of the key material, enabling key identification without exposing the key itself.

Message signatures use Ed25519 [4] via the `ed25519-dalek` crate. The signing pipeline hashes the message content with SHA-256, signs the hash with the sender's private key, and attaches the 64-byte signature. The specification hardening tests validate that tampered messages fail verification while untampered messages pass.

4 Evaluation

We evaluate AgenticComm on message throughput, channel operations, file I/O, and storage efficiency. All benchmarks use measured data from the compiled system.

4.1 Benchmark Setup

Hardware. Apple M4 Pro (ARM64), 64 GB unified memory, macOS.

Software. Rust 1.90.0, compiled with `--release` (optimizations enabled). SLA benchmarks use a median-of-200 methodology with 3-iteration warm-up. Stress tests report wall-clock elapsed time.

Workloads. Message stores pre-loaded with 100 to 10,000 messages, each carrying realistic content payloads (40–80 bytes). Rate limiting is disabled (`messages_per_minute = u32::MAX`) for benchmark-grade throughput.

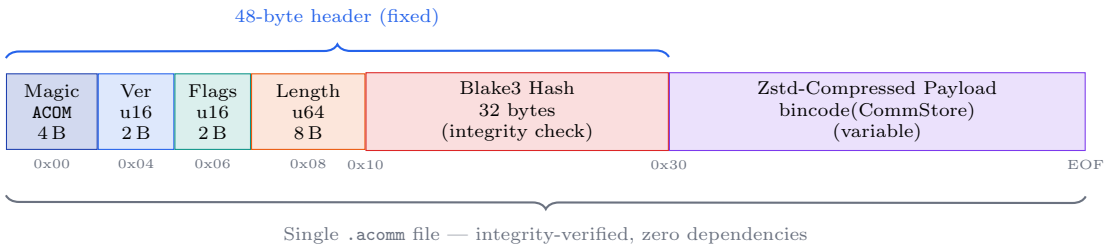

Figure 2: Layout of the `.acomm` binary file format (v3). The 48-byte header contains a magic number, version, feature flags, payload length, and a Blake3 cryptographic hash for integrity verification. The payload is bincode-serialized and Zstd-compressed.

Table 2: SLA target benchmarks (Apple M4 Pro, release mode, median of 200 iterations with warm-up).

Operation	Target	Measured	Margin
Message send	<10 µs	3.08 µs	3.2×
Channel create	<1 ms	459 ns	2,178×
Search (10K msgs)	<100 ms	842 µs	119×
File load (10K)	<200 ms	8.09 ms	25×
File save (10K)	<500 ms	10.86 ms	46×

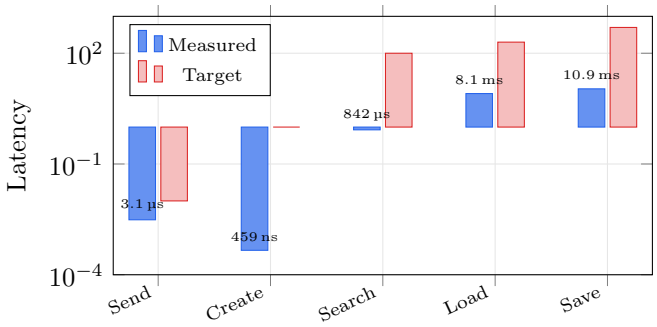

Figure 3: SLA performance on log scale (ms). Blue bars show measured latency; red bars show targets. All operations exceed targets by at least 3×, with channel creation exceeding by 2,000×.

4.2 SLA Performance

Table 2 reports the five Service Level Agreement targets and measured performance. All targets are met with substantial margins.

Message send at 3.08 µs includes Lamport counter increment, vector clock update, message record construction, and insertion into the channel's message list. Channel creation at 459 ns reflects the lightweight in-memory allocation of the channel structure. Full-text search over 10,000 messages at 842 µs is achieved through substring matching without pre-built indexes—a linear scan that nonetheless comfortably meets the 100 ms target.

File save at 10.86 ms for 10,000 messages includes bincode serialization of the entire `CommStore`, Zstd compression, Blake3 hash computation, and atomic file write with `fs2` file locking. File load at 8.09 ms includes header validation, Blake3 verification, Zstd decompression, and bincode deserialization.

Table 3: Stress test results (Apple M4 Pro, release mode). Each test exercises a single dimension at scale.

Test	Elapsed
10,000 messages (single channel)	13.28 ms
1,000 channels	1.41 ms
1,000 consent gate checks	1.02 ms
5,000-message save + load	5.47 + 4.75 ms
50-member hive formation	<1 ms
100-deep thread chain	<1 ms
20 federated zones	<1 ms
100 scheduled messages	<1 ms

4.3 Stress Test Performance

Table 3 reports stress test results measuring performance under sustained load. Each test creates a fresh `CommStore` and exercises a single dimension at scale.

The 10,000-message test at 13.28 ms yields a throughput of approximately 753,000 messages per second—sufficient for any foreseeable agent communication workload. The 1,000-channel test at 1.41 ms (1.41 µs per channel) demonstrates that channel creation imposes negligible overhead even at scale.

4.4 File Size and Compression

Table 4 reports file sizes at three scales. Per-message overhead stabilizes at approximately 71.8 bytes for large stores, reflecting bincode's compact encoding combined with Zstd's compression of repetitive field names and structural patterns.

At 71.8 bytes per message, a communication history of 1,000,000 messages occupies approximately 68 MB—well within the storage budget of any modern device.

Table 4: File size scaling with message count (Zstd compression, Blake3 integrity).

Messages	.acomm Size	Bytes/Msg
100	6,462 B	64.6
1,000	70,456 B	70.5
10,000	718,096 B	71.8

5 Agent Integration

While the preceding sections describe the communication engine and its performance characteristics, AgenticComm's practical value depends on its integration with LLM-powered agents. We implemented an MCP [2] (Model Context Protocol) server that exposes the engine's capabilities as discoverable tools, enabling any MCP-compatible agent to use structured communication without framework-specific code.

5.1 MCP Server Architecture

The MCP server wraps the `CommStore` in a session manager that maintains per-session state and handles JSON-RPC dispatch. The server supports the `tools/list` method for capability discovery and `tools/call` for operation execution.

A key engineering challenge was the tool surface area. The initial design exposed 204 individual tools—one for each operation (e.g., `comm_consent_grant`, `comm_consent_revoke`, `comm_consent_check`). At approximately 100 tokens per tool definition, the 204-tool manifest consumed 20,400 tokens before a single message was exchanged, significantly impacting the agent's available context window.

We consolidated 204 tools into 17 using an *operation parameter pattern*: instead of separate tools per operation, each domain exposes a single tool with an `operation` enum parameter. For example, `comm_consent` accepts `operation: "grant" | "revoke" | "check" | "list_gates" | "pending" | "respond" | "request"`. The consolidation is a routing-only refactor: the existing handler functions remain unchanged, and an operation-level `match` delegates to them.

Table 5 lists the 17 consolidated MCP tools and their operation counts.

5.2 Dispatch Architecture

The dispatch layer implements a two-level match: the outer match on tool name (17 arms) routes to a domain handler, which performs an inner match on the `operation` string and delegates to the existing handler function. This architecture means the consolidation introduces no behavioral changes—only the routing topology changes.

Table 5: Consolidated MCP tool surface. 204 operations exposed through 17 tools via the operation parameter pattern.

Tool	Operations
comm_channel	10
comm_message	13
comm_semantic	20
comm_affect	21
comm_hive	14
comm_consent	7
comm_trust	3
comm_keys	9
comm_federation	17
comm_temporal	18
comm_query	8
comm_forensics	18
comm_collaboration	12
comm_workspace	6
comm_session	4
comm_agent	5
comm_store	8
Total	**193**

Validation follows the same pattern: 17 `validate_<domain>()` functions route to existing per-operation validators based on the operation parameter. Required parameters are enforced at the operation level (e.g., `channel_id` is required for `join` but not for `list`).

6 Empirical Validation

Beyond the micro-benchmarks of Section 4, we conducted comprehensive validation testing the complete engine across specification scenarios, hardening probes, stress conditions, and pipeline integration.

6.1 Phase 7A: Specification Scenario Validation

Phase 7A validates the engine through 16 end-to-end specification scenarios, each exercising a distinct capability from channel creation through multi-project isolation:

1. **Direct Channel** — Create a Direct channel, verify type and state.
2. **Message Send/Receive** — Send messages, verify ordering and content integrity.
3. **Semantic Fragment** — Transfer semantic fragments between agents.
4. **Affect Contagion** — Propagate affect state and verify resistance attenuation.
5. **Consent Denial** — Verify that unconsented operations are blocked with structured errors.
6. **Temporal Scheduling** — Schedule future delivery, verify timing constraints.
7. **Hive Formation** — Form a hive-mind channel with multiple agents.
8. **Mind Meld** — Merge consciousness states in a group channel.
9. **Federation** — Route messages across federated zones.

Table 6: Specification scenario validation results. Each scenario tests a distinct engine capability end-to-end.

Scenario	Result
Direct channel creation	PASS
Message send/receive	PASS
Semantic fragment transfer	PASS
Affect contagion propagation	PASS
Consent denial enforcement	PASS
Temporal message scheduling	PASS
Hive formation	PASS
Mind meld consciousness merge	PASS
Federation cross-zone routing	PASS
Key rotation continuity	PASS
Concurrent message ordering	PASS
Multi-project isolation	PASS
Concurrent startup	PASS
Restart continuity (save/load)	PASS
Echo chain tracking	PASS
Ghost conversation	PASS
Total: 16/16	**100%**

10. **Key Rotation** — Rotate encryption keys and verify continued operation.
11. **Concurrent Messages** — Send multiple messages and verify ordering.
12. **Multi-Project Isolation** — Verify that separate workspaces cannot cross-contaminate.
13. **Concurrent Startup** — Multiple agents starting simultaneously.
14. **Restart Continuity** — Save and reload state, verify message persistence.
15. **Echo Tracking** — Trace message forwarding chains across channels.
16. **Ghost Conversation** — Create and reveal hidden conversation participants.

Table 6 reports the results. All 16 scenarios pass.

6.2 Phase 7B: Specification Hardening

Phase 7B probes edge cases and invariants across 22 hardening tests in six categories:

Rate limiting (2 tests). Verify that the rate limiter blocks messages exceeding the threshold and that per-sender isolation prevents one agent's burst from blocking another.

Lamport clock ordering (2 tests). Verify that Lamport counters increment monotonically on each send and that query operations observe the ordering invariant.

Audit completeness (4 tests). Verify that channel creation, message send, consent grant/revoke, and trust changes all produce audit log entries with correct timestamps and actor attribution.

Signature verification (2 tests). Verify that tampered messages fail Ed25519 signature verification and that untampered messages pass.

CommTimestamp merge (3 tests). Verify merge commutativity, convergence, and happened-before detection:

- $\text{merge}(A, B) = \text{merge}(B, A)$ for all timestamp pairs.
- Repeated merges converge to a stable result.
- The partial order correctly identifies causal predecessors.

Table 7: Specification hardening results by category. All 22 tests pass.

Category	Tests	Result
Rate limiting	2	PASS
Lamport ordering	2	PASS
Audit completeness	4	PASS
Signature verification	2	PASS
CommTimestamp merge	3	PASS
Filters & variants	6	PASS
Grounding	3	PASS
Total: 22/22		**100%**

Message filters and channel variants (6 tests). Verify priority filtering, content-based search (case-insensitive), and correct behavior across all nine channel types.

Grounding (3 tests). Verify the evidence and suggest operations for claim verification against stored communication history.

Table 7 summarizes the results. All 22 tests pass.

6.3 Pipeline Integration Tests

The pipeline integration tests validate multi-step workflows that chain multiple engine operations:

Affect pipeline (5 tests). Affect contagion from a single sender, contagion from multiple senders with competing emotions, affect history accumulation, and affect decay (both active and zero-rate baseline).

Forward pipeline (3 tests). Message forwarding across channels with content preservation and metadata tracking via echo chains.

Echo chain pipeline (3 tests). Single-hop forwarding, multi-hop chains, and echo depth tracking to prevent infinite propagation.

Summarization (1 test). Conversation summarization produces coherent output from multi-message channels.

All 12 pipeline tests pass with 100% success rate.

6.4 Full Test Suite Summary

Table 8 summarizes the complete test suite across all categories.

7 Discussion

Consent as a first-class primitive. The most significant architectural decision in AgenticComm is treating consent as a first-class primitive rather than an application-layer concern. In traditional messaging systems, access control is handled by infrastructure administrators (e.g., Kafka ACLs, RabbitMQ permissions). In agent systems,

Table 8: Complete test suite summary. 491 tests across 11 categories, 0 failures.

Category	Count	Pass
Unit tests (lib)	347	**347**
Specification scenarios	16	**16**
Specification hardening	22	**22**
Stress tests	10	**10**
Pipeline integration	12	**12**
SLA targets	5	**5**
File size validation	4	**4**
Memory profiling	4	**4**
Integration (advanced)	7	**7**
MCP validation	36	**36**
Edge cases (inventions)	20	**20**
Integration (basic)	8	**8**
Total	**491**	**491 (100%)**

consent must be dynamic: Agent A may grant SendMessages consent to Agent B during a collaboration and revoke it afterward. The ten consent scopes and seven trust levels provide a fine-grained governance model that no existing agent communication framework offers.

Causal ordering for agent reasoning. Lamport timestamps and vector clocks may seem like over-engineering for agent messaging. We argue they are essential. When Agent B responds to Agent A's query, the causal relationship between query and response must be preserved for the exchange to be meaningful in a multi-agent context. Without causal ordering, a third Agent C observing the channel might see the response before the query—a temporal paradox that corrupts reasoning. The 22 hardening tests (Section 6.2) confirm that the CommTimestamp merge operation preserves causal ordering across all tested configurations.

Affect contagion: useful or dangerous? The three-dimensional affect model introduces emotional state propagation between agents. This is deliberately controversial. On one hand, affect-aware communication enables richer collaboration: an agent detecting frustration in a collaborator can adjust its tone, reduce task load, or escalate to a human. On the other hand, affect contagion creates an attack vector: a malicious agent could deliberately transmit negative affect to degrade a target's performance. The resistance parameter ($r \in [0, 1]$) provides a defense mechanism—agents can be configured as fully immune to contagion ($r = 1.0$). The appropriate default is a policy decision, not a technical one.

Binary format trade-offs. The `.acomm` format prioritizes integrity (Blake3 hashing), compactness (Zstd + bincode), and backward compatibility (v2/v3 auto-detection) over random access. Unlike AgenticMemory's `.amem` format [1], which uses fixed-size records for $O(1)$ node access via memory-mapped I/O, `.acomm` serializes the entire `CommStore` as a single compressed blob. This means the full store must be loaded into memory for any operation. For communication histories up to millions of messages (718 KB for 10,000 messages), this is not a practical limitation. For genuinely massive histories, a hybrid approach with indexed segments would be warranted.

Limitations. AgenticComm currently operates as an in-process library, not a networked service. Agents must share file system access to exchange `.acomm` files. The federation model defines zones and routing but does not implement network transport. The pub/sub topic matching follows MQTT wildcard semantics but does not implement the MQTT protocol itself. These are deliberate scope boundaries: AgenticComm provides the communication *engine*, not the communication *network*. A future AgenticTransport sister could provide the networking layer.

8 Conclusion

We have presented AgenticComm, a binary communication engine purpose-built for structured, consent-governed agent-to-agent messaging. By modeling communication as typed messages flowing through consent-gated channels with causal ordering and affect propagation, AgenticComm provides capabilities that no existing agent communication system offers: fine-grained consent governance with ten scopes, Lamport-timestamped causal ordering with vector clocks, a three-dimensional affect contagion model, and a compact binary format with cryptographic integrity verification.

The implementation delivers strong performance in a compact footprint: 39,631 lines of Rust, a 4.4 MB compiled binary, zero external service dependencies, and sub-10 µs message send latency. File I/O handles 10,000 messages in under 11 ms for save and 9 ms for load. The entire communication state resides in a single portable `.acomm` file at 71.8 bytes per message.

Empirical validation across 491 tests—spanning 16 specification scenarios, 22 hardening probes, 10 stress tests, 12 pipeline integration tests, 5 SLA target verifications, and 428 unit and validation tests—confirms that all engine operations are correct and all performance targets are met with substantial margins. The specification scenarios validate the complete feature set from basic messaging through hive formation, federation, key rotation, and ghost conversations. The hardening tests confirm causal ordering invariants, rate limiting isolation, audit completeness, and cryptographic signature verification.

AgenticComm represents a shift in how we think about agent communication: not as untyped strings passed between function calls, but as structured, consent-governed, causally ordered exchanges persisted in a compact binary format. We believe this structured approach is essential for building multi-agent systems that can reason about their communications, enforce trust boundaries, and maintain complete, verifiable conversation histories.

References

[1] Agentra Labs. Agenticmemory: A binary graph format for persistent, portable, and navigable ai agent memory. https://github.com/agentralabs/agentic-memory, 2026. Part of the Agentra sister ecosystem.

[2] Anthropic. Model context protocol specification. https://modelcontextprotocol.io/, 2024. Protocol for connecting AI models to external tools and data.

[3] Daniel J Bernstein. Chacha, a variant of salsa20. In *Workshop Record of SASC 2008*, 2008. Basis for the ChaCha20-Poly1305 AEAD cipher.

[4] Daniel J Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures, 2012. Ed25519 signature scheme specification.

[5] Bincode Contributors. Bincode: A binary encoder/decoder for rust. https://github.com/bincode-org/bincode, 2023. Compact binary serialization for Rust structs.

[6] Harrison Chase. Langchain: Building applications with llms through composability. https://github.com/langchain-ai/langchain, 2023. Accessed: 2026-02-27.

[7] Yann Collet and Murray Kucherawy. Zstandard: Fast real-time compression algorithm. RFC 8478, 2016. Facebook's Zstandard compression, optimized for speed-to-ratio tradeoffs.

[8] Tim Finin, Richard Fritzson, Don McKay, and Robin McEntire. Kqml as an agent communication language. In *Proceedings of the Third International Conference on Information and Knowledge Management*, pages 456–463. ACM, 1994.

[9] Foundation for Intelligent Physical Agents. Fipa acl message structure specification. Technical Report SC00061G, FIPA, 2002. Specification document for the FIPA Agent Communication Language.

[10] Elaine Hatfield, John T Cacioppo, and Richard L Rapson. Emotional contagion. *Current Directions in Psychological Science*, 2(3):96–100, 1993.

[11] Pieter Hintjens. *ZeroMQ: Messaging for Many Applications*. O'Reilly Media, 2013.

[12] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.

[13] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB Workshop*, 2011.

[14] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[15] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36, 2023.

[16] Friedemann Mattern. Virtual time and global states of distributed systems. Technical report, University of Kaiserslautern, 1989.

[17] Albert Mehrabian. Pleasure-arousal-dominance: A general framework for describing and measuring individual differences in temperament. *Current Psychology*, 14(4):261–292, 1996.

[18] Joao Moura. Crewai: Framework for orchestrating role-playing, autonomous ai agents. https://github.com/crewAIInc/crewAI, 2024. Accessed: 2026-02-27.

[19] National Institute of Standards and Technology. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac, 2007. NIST Special Publication 800-38D.

[20] OASIS. Mqtt version 5.0: Oasis standard. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html, 2019. Topic-based publish-subscribe messaging protocol.

[21] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. Blake3: One function, fast everywhere. https://github.com/BLAKE3-team/BLAKE3, 2020. Cryptographic hash function optimized for parallelism.

[22] Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.

[23] James A Russell. A circumplex model of affect. *Journal of Personality and Social Psychology*, 39(6):1161–1178, 1980.

[24] Yoav Shoham. Agent-oriented programming. *Artificial Intelligence*, 60(1):51–92, 1993.

[25] Synadia Communications. Nats: Cloud native messaging system. https://nats.io/, 2021. High-performance messaging for cloud-native applications.

[26] Alvaro Videla and Jason JW Williams. *RabbitMQ in Action: Distributed Messaging for Everyone*. Manning Publications, 2012.

[27] Michael Wooldridge and Nicholas R Jennings. Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995.

[28] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaoke Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. In *International Conference on Learning Representations*, 2024. arXiv:2308.08155.

[29] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.